Основные архитектуры CNN

Зачем нужны новые архитектуры CNN?

На прошлой лекции мы разобрали базовую идею свёрточных сетей и увидели, почему они принципиально лучше MLP подходят для обработки изображений. Типичный конвейер выглядел так:

$$\text{Conv} \to \text{ReLU} \to \text{Pooling} \to ... \to \text{Flatten} \to \text{Linear} \to \text{Softmax} \tag{1}$$

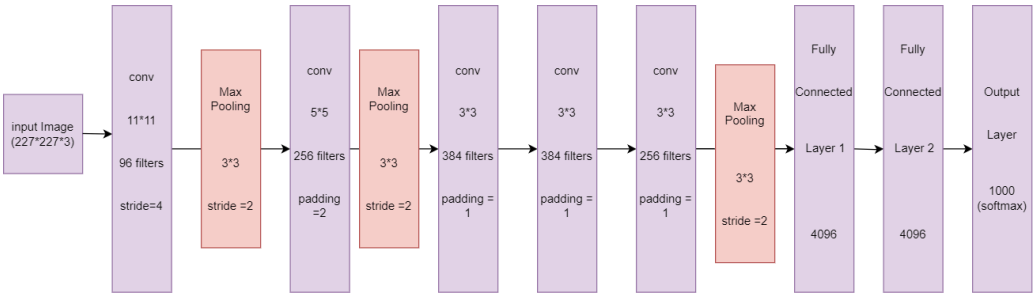
Однако довольно быстро стало понятно, что одного этого шаблона недостаточно. У классических CNN обнаружилось несколько существенных проблем:

1. **Трудность обучения глубоких сетей.** При увеличении числа слоёв ухудшается прохождение градиента.
2. **Потеря информации.** Пулинг и последовательные сжатия уменьшают пространственную точность признаков.
3. **Невозможность эффективно передавать ранние признаки в глубокие слои.**

Именно попытки преодолеть эти ограничения привели к появлению ряда знаковых архитектур: AlexNet, VGG, ResNet и U-Net. Каждая из них решала свою часть общей проблемы и в итоге сильно повлияла на развитие компьютерного зрения.

AlexNet

Одной из первых по-настоящему прорывных свёрточных архитектур стала **AlexNet**. Она была предложена в 2012 году Алексом Крижевским, Ильёй Суцкевером и Джеффри Хинтоном и победила в соревновании ImageNet, фактически начав современную эпоху deep learning в компьютерном зрении.



AlexNet Architecture

Рис. 1. Схема архитектуры AlexNet

Основная проблема того времени состояла в том, что глубокие сети было тяжело обучать. Для её решения в AlexNet удачно совместили несколько идей:

- функцию активации **ReLU**, которая существенно ускоряла обучение по сравнению с сигмоидой и tanh;
- **Dropout** в полносвязных слоях как способ борьбы с переобучением;
- **Data Augmentation**, то есть искусственное увеличение обучающей выборки за счёт преобразований изображений.

> **Замечание**
>
> В историческом смысле AlexNet важна не только как конкретная архитектура, но и как демонстрация того, что глубокие CNN действительно можно обучать на больших данных.

Архитектура AlexNet содержала несколько свёрточных слоёв, после которых шли очень крупные полносвязные слои. По современным меркам это решение выглядит тяжёлым, но на тот момент оно оказалось исключительно эффективным.

Ограничения AlexNet

Несмотря на успех, у AlexNet были заметные недостатки:

- очень большие FC-слои, дававшие огромное число параметров;
- не самая аккуратная и унифицированная структура;
- отсутствие ясного принципа, как системно масштабировать архитектуру по глубине.

Тем не менее именно AlexNet показала, что комбинация правильной архитектуры, хорошей оптимизации и мощного железа может дать качественный рывок в задачах зрения.

VGG

После AlexNet возник естественный вопрос: а можно ли построить более простую и системную архитектуру, просто делая сеть глубже? Одним из самых известных ответов стала **VGG**, предложенная группой Visual Geometry Group из Оксфорда в 2014 году.

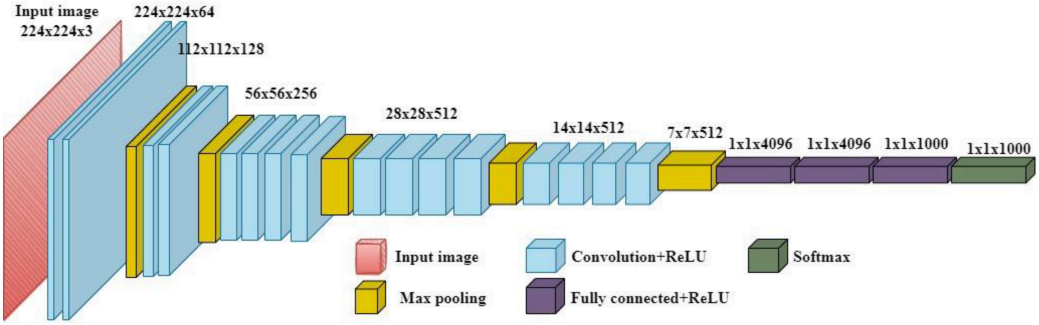


Рис. 2. Пример глубокой архитектуры семейства VGG

Главная идея VGG состояла в максимальном упрощении строительных блоков. Вместо смеси разных по размеру фильтров архитектура строилась почти полностью на последовательностях свёрток 3×3 с последующим pooling.

Почему именно 3×3?

Малые ядра дают сразу несколько преимуществ:

- меньше параметров по сравнению с большими фильтрами;
- возможность наращивать глубину сети;
- постепенный рост **receptive field**, то есть области входного изображения, влияющей на конкретный нейрон.

Например, одна свёртка 3×3 даёт receptive field 3×3, две подряд — уже примерно 5×5, а три подряд — около 7×7. Тем самым несколько маленьких свёрток способны заменить одну большую, сохранив выразительность и даже добавив дополнительные нелинейности между слоями.

Общая схема VGG

Сеть строится из повторяющихся блоков вида

$$[\text{Conv } 3 \times 3 \to \text{ReLU}] \times N \to \text{Pooling} \tag{2}$$

При движении в глубину число каналов увеличивается, а пространственный размер уменьшается. Типичная схема числа каналов выглядит так:

$$64 \to 128 \to 256 \to 512 \to 512 \tag{3}$$

В конце по-прежнему располагались полносвязные слои и softmax-классификатор.

Недостатки VGG

Хотя VGG стала очень популярной благодаря простоте и понятности, у неё были серьёзные минусы:

1. Огромное количество параметров.
2. Высокая вычислительная стоимость.
3. Ограниченная масштабируемость по глубине.

Другими словами, VGG сделала архитектуру более чистой и системной, но не решила фундаментальную проблему обучения действительно очень глубоких сетей. Эту проблему удалось существенно продвинуть архитектуре ResNet.

ResNet

При дальнейшем углублении CNN выяснилась парадоксальная вещь: добавление новых слоёв должно было бы только расширять класс представимых функций, но на практике оптимизация становилась хуже, и качество могло даже падать. Это связано, в частности, с затуханием градиента и общей сложностью оптимизации.

Идея **ResNet** состоит в том, чтобы учить не само преобразование $f(x)$, а **остаток** относительно входа:

$$y = f(x) + x \tag{4}$$

Здесь x пробрасывается дальше напрямую, а сеть обучает только то, что нужно к нему добавить. Такой механизм называется **skip connection** или остаточная связь.

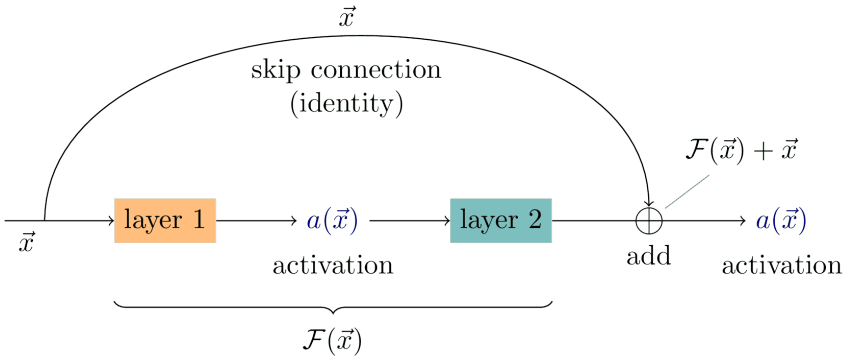


Рис. 3. Иллюстрация skip connection

Интуитивно это полезно по нескольким причинам.

- Если оптимальное преобразование близко к тождественному, сети легче выучить маленькую поправку, чем полностью новую сложную функцию.
- Градиент получает более короткий путь назад по сети, поэтому глубокие модели обучаются стабильнее.
- Сходимость становится быстрее, а увеличение глубины перестаёт так сильно ломать обучение.

Архитектура ResNet была представлена в 2015 году и быстро стала стандартом де-факто. Существуют разные варианты — ResNet-18, ResNet-34, ResNet-50, ResNet-101, ResNet-152 и другие. Они отличаются в основном количеством блоков и внутренним устройством остаточных модулей.

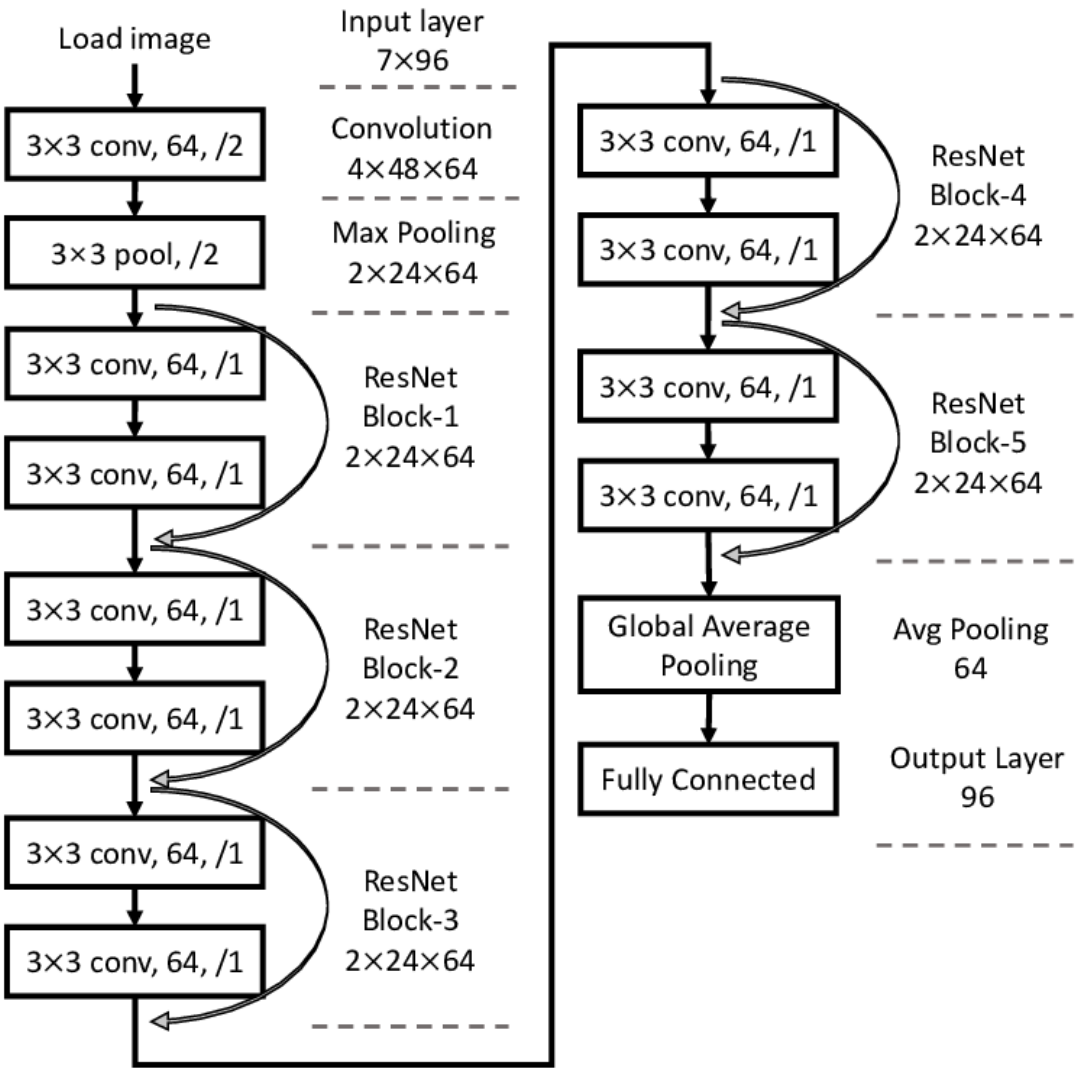


Рис. 4. Схематичное устройство семейства ResNet

ResNet принесла победу в соревновании ImageNet 2015 года и в более широком смысле изменила подход к проектированию глубоких сетей. Сегодня skip connections используются практически повсеместно, в том числе далеко за пределами классических CNN.

U-Net и задача сегментации

До сих пор мы в основном говорили о классификации изображений, где на выходе нужно получить один класс для всей картинки. Но существует и другая важная задача — **сегментация**. В ней необходимо предсказать метку класса для **каждого пикселя** изображения.

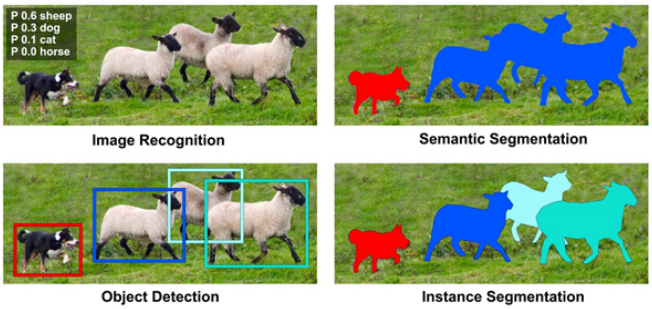


Рис. 5. Пример постановки задачи сегментации

Для такой задачи обычный классификатор не подходит: он умеет сжимать изображение до вектора признаков, но не умеет восстанавливать пространственную карту ответов. Поэтому была предложена архитектура **U-Net**.

Общая идея U-Net

U-Net состоит из двух больших частей:

- **Encoder** — постепенно уменьшает пространственный размер изображения и увеличивает число каналов, извлекая всё более абстрактные признаки;
- **Decoder** — восстанавливает пространственное разрешение и строит карту предсказаний для пикселей.

В типичном энкодере повторяется блок

$$\text{Conv} \to \text{ReLU} \to \text{Conv} \to \text{ReLU} \to \text{Pooling} \tag{5}$$

В самой глубокой части сети, которую иногда называют **bottleneck**, находятся наиболее абстрактные представления с самым большим receptive field. Затем декодер постепенно поднимает разрешение обратно, применяя upsampling и свёртки.

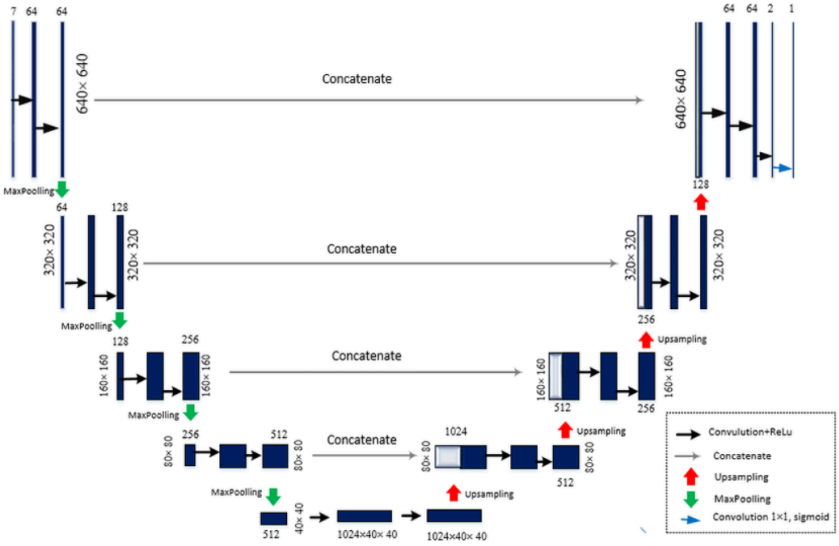


Рис. 6. Классическая схема архитектуры U-Net

Форма этой сети напоминает букву U, откуда и произошло её название.

Почему U-Net работает хорошо?

Ключевая особенность U-Net — наличие skip connections между энкодером и декодером на одинаковых уровнях разрешения. Благодаря этому декодер получает не только глубокие абстрактные признаки, но и более точную пространственную информацию из ранних слоёв. Это особенно важно для точного выделения границ объектов.

Функции потерь для сегментации

Поскольку сегментация оценивает качество перекрытия областей, здесь часто используют специальные функции потерь.

Dice Loss

Пусть A — множество пикселей, предсказанных моделью как принадлежащие объекту, а B — истинная маска. Тогда коэффициент Dice равен

$$\text{Dice coef} = \frac{2\,|A \cap B|}{|A| + |B|} \tag{6}$$

а соответствующая функция потерь задаётся как

$$\text{Dice loss} = 1 - \text{Dice coef}. \tag{7}$$

IoU

Другой популярный показатель — **Intersection over Union**:

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|} \tag{8}$$

а значит, функция потерь может быть записана как

$$\text{IoU loss} = 1 - \text{IoU}. \tag{9}$$

IoU обычно считают более строгой метрикой, так как она сильнее штрафует лишние и пропущенные области. Эти две метрики тесно связаны:

$$\text{Dice} = \frac{2\ \text{IoU}}{1 + \text{IoU}} \tag{10}$$

U-Net и её многочисленные модификации до сих пор остаются одним из основных инструментов в задачах медицинской сегментации, обработки спутниковых снимков и других областях, где необходимо предсказывать структуру изображения на уровне пикселей.